

How to adapt extern flash

This chapter introduces how to adapt 4-wire extern SPI NOR flash.

Add new type

Add your specific flash configuration in the file:

```
components\ql-kernel\drivers\quec_spi_nor_flash_prop.c
```

```
components > ql-kernel > drivers > C quec_spi_nor_flash_prop.c > ...
29  /* 4-line SPI NOR Flash supported model list,can add by yourself*/
30  static quec_spi_nor_flash_info_s quec_spi_nor_flash_props[] =
31  {
32  //You can cut out it by defining macros
33  //define DISABLE_GD25LE64E_C
34  #ifndef DISABLE_GD25LE64E_C
35  {
36      // GD25LE64E
37      // GD25LE64C
38      .mid = 0x1760c8,
39      .capacity = 0x800000,
40      .mode = QUEC_NOR_FLASH_MODE_16BIT,
41      .mount_start_addr = 0x00000000,
42      .mount_size = 0x800000,
43      .pb_size = 0x200,
44      .eb_size = 0x1000,
45  },
46  #endif
47  #ifndef DISABLE_XM25QU64B
48  {
49      // XM25QU64B
50      .mid = 0x175020,
51      .capacity = 0x800000,
52      .mode = QUEC_NOR_FLASH_MODE_8BIT,
53      .mount_start_addr = 0x00000000,
54      .mount_size = 0x800000,
55      .pb_size = 0x200,
56      .eb_size = 0x1000,
57  },
```

The array definition of quec_spi_nor_flash_props[] is in:

```
components\ql-kernel\inc\quec_spi_nor_flash_prop.h
```

```
components > ql-kernel > inc > h quec_spi_nor_flash_prop.h > ...
36
37 typedef struct
38 {
39     unsigned int mid;           //Product ID, the 3-byte product ID read with 9
40     int64 capacity;            //flash capacity, 32Mbit:0x400000 64Mbit:0x8000
41     quec_spi_nor_flash_status_mode_e mode; //FLASH mode
42
43     unsigned int mount_start_addr; //flash start addr to mount the file system,0x0
44     int32 mount_size;              //the amount of space used to mount the file sy
45     uint32_t pb_size;              //flash pb block size, general use 0x200,in spe
46     uint32_t eb_size;              //flash erase block size, general use 0x1000(4K
47 }quec_spi_nor_flash_info_s;
48
```

How to get the Key info from datasheet

Example-1: GD25LQ64E

The screenshots of GD25LQ64E's datasheet are as below shown:

TABLE OF ID DEFINITIONS
GD25LQ64E

Operation Code	MID7-MID0	ID15-ID8	ID7-ID0
9FH	C8	60	17
90H	C8		16
ABH			16

The Write **Status** Register (WRSR) command allows the user to change the values of the Block Protect (BP4, BP3, BP2, BP1, and BP0) bits, to define the size of the area that is to be treated as read-only. The Write **Status** Register (WRSR) command also allows the user to set or reset the **Status** Register Protect (SRP1 and SRP0) bits in accordance with the Write Protect (WP#) signal. The **Status** Register Protect (SRP1 and SRP0) bits and Write Protect (WP#) signal allow the device to be put in the Hardware Protected Mode. The Write **Status** Register (WRSR) command is not executed once the Hardware Protected Mode is entered.

Figure 8. Write **Status** Register Sequence Diagram (SPI)

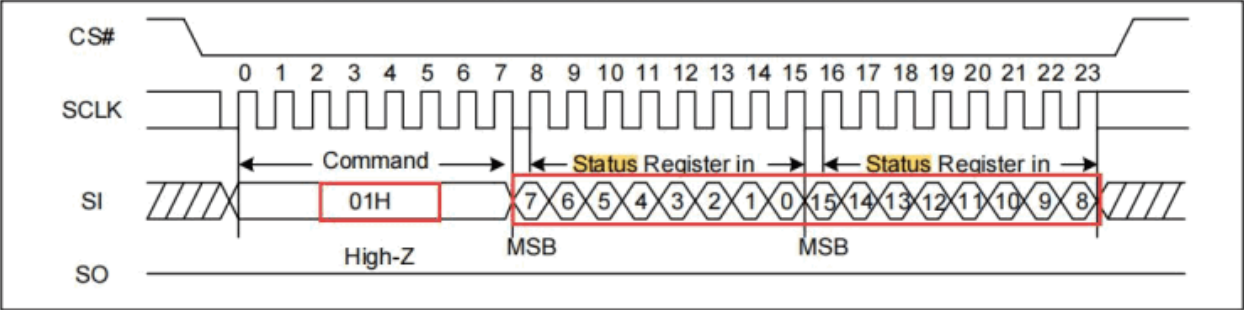
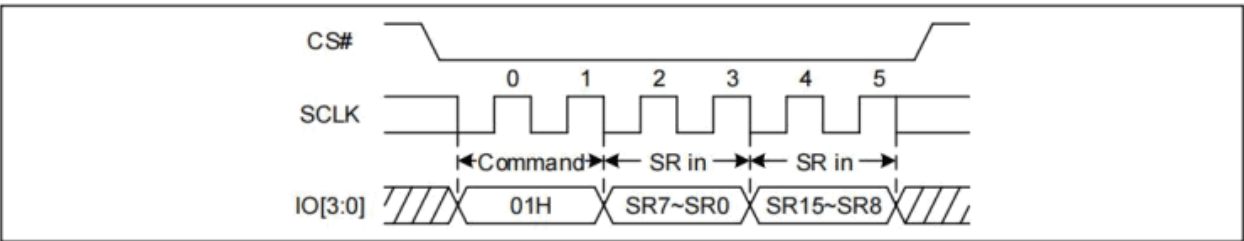


Figure 9. Write **Status** Register Sequence Diagram (QPI)



We can find the **mid** is **0x1760c8** by **9FH** cmd in the datasheet.

And we can know that the **Write Status Register (WRSR)** can write 16-bits at a time by **01H** cmd. So the parameter of **mode** should be set to **QUEC_NOR_FLASH_MODE_16BIT**.

Example-2: XM25QU64B

The screenshots of XM25QU64B's datasheet are as below shown:

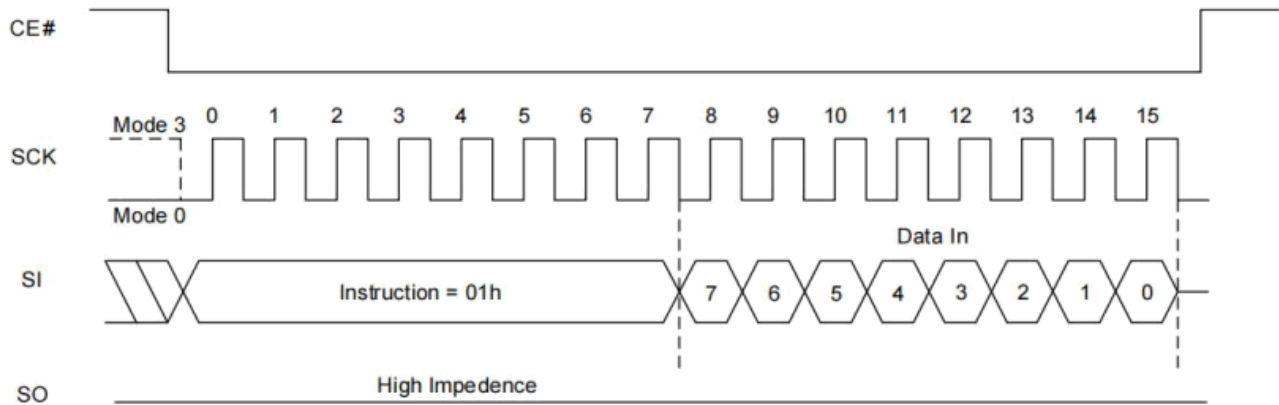
Table 8.5 Product Identification

Manufacturer ID		(MF7-MF0)	
XMC Serial Flash		20h	
Instruction	ABh	90h	9Fh
Device Density	Device ID (ID7-ID0)		Memory Type + Capacity (ID15-ID0)
64Mb	16h		5017h

8.17 WRITE STATUS REGISTER OPERATION (WRSR, 01h)

The Write Status Register (WRSR) instruction allows the user to enable or disable the block protection and Status Register write protection features by writing "0"s or "1"s into the non-volatile BP3, BP2, BP1, BP0, QE, and SRWD bits. Also WRSR instruction allows the user to disable or enable quad operation by writing "0" or "1" into the non-volatile QE bit.

Figure 8.28 Write Status Register Sequence In SPI Mode



We can find the `mid` is `0x175020` by `9FH` cmd in the datasheet.

And we can know that the **Write Status Register (WRSR)** can only write 8-bits at a time by `01H` cmd. So the parameter of `mode` should be set to `QUEC_NOR_FLASH_MODE_8BIT`.